

Training recipes and graph-native learning

ar088 · 14 August 2026 · Draft

Contents

1. Developer guide
2. Training is separate from the graph
3. Current boundary
4. API reference

Developer guide

Training is separate from the graph

`TrainSpec` describes standard learning choices without embedding a training loop in graph structure. It refers to named outputs and parameters in one network.

```
from tools.snnlang import training

recipe = training.TrainSpec(
    objectives=[training.CrossEntropy(
        prediction=readout,
        target="digit",
    )],
    parameter_groups=[training.ParameterGroup(
        name="feedforward",
        parameters=readout.parameters,
        lr=1e-3,
    )],
    optimizer=training.AdamW(weight_decay=1e-4),
    epochs=20,
)

bundle = snn.compile(net, training=recipe)
```

The recipe vocabulary includes objectives, named parameter groups, learning rates, frozen parameters, optimizer settings, regularizers, stop-gradient boundaries, epoch count, gradient clipping, and surrogate configuration. Every graph parameter must belong to exactly one group. Trainable groups require a positive finite learning rate; frozen groups require zero. Compilation records canonical resolved trainable and frozen sets plus a per-parameter learning-rate map so backends never infer ownership from naming conventions.

Current boundary

Training recipes compile, and a narrow one-layer MNIST PING recipe can be translated into the legacy trainer. Graph-native training is not implemented. A graph training request fails explicitly and never falls back silently to legacy execution.

The complete implementation also needs deterministic initialization, differentiable-reachability checks, recurrent trainability, variable-rate training, spike-budget regularization, selected and final checkpoints, optimizer-state replay, exact resume, and layered legacy-versus-graph conformance tests.

API reference

TrainSpec

```
training.TrainSpec(  
    objectives,  
    parameter_groups,  
    optimizer,  
    regularizers=(),  
    stop_gradients=(),  
    epochs=1,  
    gradient_clip=None,  
    surrogate=training.FastSigmoid(slope=1.0),  
    presentation_duration=200 * snn.ms,  
)
```

The specification is data only. `compile` verifies referenced outputs, parameters, regularizer signals, and stop-gradient signals before writing `training.json`.

The collection's supported surrogate is `training.FastSigmoid(slope=1.0)`. Its slope must be positive and finite. Voltage-gradient dampening is declared per spiking population as `voltage_grad_dampen`; it is also resolved into `training.json` so training provenance contains the complete backward contract. Dampening factors must be positive and finite. The narrow legacy adapter accepts fast-sigmoid slopes and a single shared dampening factor, and rejects richer unsupported combinations explicitly.

Presentation duration is a positive millisecond quantity independent of graph timestep, but it must resolve to an integer number of steps. `SpikeBudgetPenalty` implements the exp022 one-sided quadratic firing-rate ceiling. It converts each population's spike count to a per-presentation population-mean rate in Hz, applies the squared hinge above the ceiling, then averages presentations and layers. This makes the term invariant to batch size, population width, hidden-layer count, and equivalent changes in presentation duration.

Objectives and parameter groups

```
training.CrossEntropy(*, prediction, target, weight=1.0) -> Objective  
training.ParameterGroup(parameters, name, lr, frozen=False)
```

`prediction` accepts a signal-like object or identifier. It must resolve to a named graph output. `target` is the external target identifier. Group identifiers are non-empty and unique; groups

are non-empty; and every parameter appears exactly once across the groups. Frozen groups use `lr=0`, while trainable groups use a positive finite rate.

Optimizer and regularization

```
training.AdamW(**config) -> Optimizer
training.UpperRatePenalty(*, signal, threshold, strength) -> Regularizer
training.SpikeBudgetPenalty(*, signals, ceiling_hz, strength) -> Regularizer
training.StopGradient.at(signal) -> StopGradient
training.FastSigmoid(*, slope=1.0) -> Spec
```

Optimizer configuration is serialized without running an optimizer. `SpikeBudgetPenalty` is the canonical multi-layer form; `UpperRatePenalty` is its single-signal compatibility spelling. `StopGradient.at` records a graph boundary by signal identifier.

Execution support

`snn.compile(net, training=recipe)` validates and serializes the recipe.

`tools.snn.execution.train(ExecutionSpec(executor="graph", ...))` currently raises `NotImplementedError`. Use the explicit legacy executor only for the supported compatibility adapter.

Next: Runtime state, checkpoints, and provenance